

Mon Projet Expense Tracker

Tout Comprendre de A a Z

Guide personnel d'apprentissage

Architecture AWS, Code C#, Interface MAUI



ESTIAM Paris - Master 2 - 2026

Table des matieres

Chapitre 1	Vue d'ensemble - C'est quoi ce projet ?
Chapitre 2	L'Architecture - Comment les pieces s'assemblent
Chapitre 3	AWS Lambda - Le Coeur du Backend
Chapitre 4	Amazon DynamoDB - La Base de Donnees NoSQL
Chapitre 5	Amazon API Gateway - La Porte d'Entree
Chapitre 6	Amazon Cognito - L'Authentification
Chapitre 7	Amazon S3 - Le Stockage des Recus
Chapitre 8	IAM - La Securite des Permissions
Chapitre 9	Le Frontend - .NET MAUI et C#
Chapitre 10	Structure du Code Frontend - Les Services et Pages
Chapitre 11	Le Monitoring - CloudWatch
Chapitre 12	Les Bugs qu'on a Rencontres et Comment on les a Resolus
Chapitre 13	Glossaire - Les Termes Importants

Chapitre 1 - Vue d'ensemble - C'est quoi ce projet ?

1.1 Le probleme que l'on resout

Dans beaucoup d'entreprises, quand un employe fait une depense professionnelle (repas avec un client, billet de train, achat de materiel), il doit en demander le remboursement. Avant le numerique, ca se faisait avec des formulaires papier, des signatures, des classeurs. Long, fastidieux, et on perd souvent les recus.

Notre projet **Expense Tracker** resout ce probleme : c'est une application complete qui permet de creer, soumettre et approuver des notes de frais de facon numerique, securisee, et tracable.

1.2 Les deux utilisateurs

Notre application a deux types d'utilisateurs avec des roles differents :

Alice (Employe) peut : Creer une note de frais (montant, categorie, description), Joindre un reçu photo, Soumettre sa note pour approbation, Voir l'historique de ses notes et leur statut.

Bob (Finance Manager) peut : Voir toutes les notes soumises par tous les employes, Approuver une note (l'employe sera rembourse), Rejeter une note avec un motif, Consulter l'historique complet.

Analogie :

Imagine une feuille de remboursement papier que tu remplis et deposes dans la boite de la comptabilite. Notre app fait exactement ca, mais en numerique : plus de papier perdu, tout est trace, et le manager peut approuver depuis n'importe ou.

1.3 Le cycle de vie d'une note de frais

Chaque note de frais passe par des etats bien definis, comme un colis qu'on suit :

DRAFT (Brouillon) : Alice vient de creer la note, elle n'est pas encore soumise. Elle peut encore la modifier.

SUBMITTED (Soumise) : Alice a clique 'Soumettre'. La note attend l'approbation de Bob.

APPROVED (Approuvee) : Bob a approuve. Alice sera remboursee.

REJECTED (Rejetee) : Bob a rejete avec un motif. Alice peut corriger et re-soumettre.

Ces transitions sont controlees cote serveur (dans nos Lambda functions). Le client ne peut pas tricher - on ne peut pas approuver une note qui n'est pas en statut SUBMITTED, par exemple.

1.4 Pourquoi AWS et pas un serveur classique ?

On aurait pu faire cette app avec un serveur Apache + PHP + MySQL comme en cours. Mais on a choisi AWS pour plusieurs raisons :

Pas de serveur a gerer : Avec AWS Lambda, on n'a pas de serveur qui tourne 24h/24. Le code s'execute seulement quand quelqu'un fait une requete.

Paieement a l'usage : On paye seulement les executions Lambda, pas un serveur qui tourne meme quand personne l'utilise.

Scalabilite automatique : Si 1000 employes soumettent des notes en meme temps, AWS s'adapte automatiquement.

Securite integree : Cognito, IAM, HTTPS - tout est deja la.

Points clés :

Notre app remplace le processus papier de notes de frais. Deux roles : employe (Alice) et finance manager (Bob). Cycle de vie : DRAFT -> SUBMITTED -> APPROVED/REJECTED. AWS offre scalabilite, securite et paiement a l'usage.

Chapitre 2 - L'Architecture - Comment les pieces s'assemblent

2.1 La vue d'ensemble

Notre application est composee de plusieurs services AWS qui communiquent entre eux. Voici le chemin complet d'une requete :

```
[Application MAUI Windows/Android]
  |
  | HTTPS + Token JWT
  v
[Amazon API Gateway] <-- verifie le token avec Cognito
  |
  | appelle la bonne fonction
  v
[AWS Lambda (C# .NET 8)] <-- execute la logique metier
  |           |
  v           v
[DynamoDB]    [Amazon S3]
(donnees)    (photos recus)

[Amazon Cognito] --> fournit les tokens JWT
[IAM Role] --> donne les permissions a Lambda
```

2.2 Ce qui se passe quand Alice clique 'Creer une note'

Voici le chemin complet, etape par etape :

1. Alice remplit le formulaire dans l'app MAUI (montant: 50 euros, categorie: REPAS, description: Dejeuner client)
2. L'app envoie une requete HTTP POST vers API Gateway avec le token JWT dans le header Authorization
3. API Gateway recoit la requete. Il extrait le token JWT et le verifie aupres de Cognito
4. Cognito confirme : 'Oui, c'est le token d'Alice, elle est dans le groupe employees'
5. API Gateway transmet la requete a la fonction Lambda CreateExpense avec les informations d'Alice
6. Lambda cree un item dans DynamoDB avec PK=USER#alice-uuid, SK=EXPENSE#nouveau-uuid, status=DRAFT
7. Lambda repond : {"expenseId": "abc123", "status": "DRAFT"}
8. API Gateway renvoie cette reponse a l'app MAUI
9. L'app affiche 'Note creee avec succes !'

Tout ca se passe en moins d'une seconde.

2.3 Le concept Serverless explique simplement

Analogie :

Taxi vs Voiture personnelle : Un serveur classique, c'est comme avoir une voiture : elle prend de la place dans le garage, elle consomme même quand tu ne l'utilises pas, tu dois faire la révision toi-même. AWS Lambda c'est comme Uber : tu commandes une voiture seulement quand tu en as besoin, tu payes uniquement le trajet, et tu n'as pas à t'occuper de la maintenance.

Points clés :

L'architecture suit le chemin : MAUI -> API Gateway -> Lambda -> DynamoDB/S3. API Gateway vérifie le token JWT avant chaque appel Lambda. Serverless = pas de serveur à gérer, paiement à l'usage uniquement.

Chapitre 3 - AWS Lambda - Le Coeur du Backend

3.1 C'est quoi une fonction Lambda ?

Analogie :

La machine distributrice : Imagine une machine distributrice. Elle ne fait rien tant que personne n'appuie sur un bouton. Quand tu appuies sur 'Coca', elle execute une action precise (distribue un Coca) et s'arrete. C'est exactement Lambda : dormant par default, il s'execute quand il recoit une requete, fait son travail, et s'arrete.

Une Lambda function, c'est un morceau de code qui s'execute en reponse a un evenement. Dans notre cas, l'evenement est une requete HTTP qui arrive sur API Gateway. Lambda recoit la requete, execute le code C#, et renvoie une reponse.

3.2 Nos 7 fonctions et leur role

CreateExpense - Creer une note de frais

Recoit : montant, categorie, description + token JWT. Fait : extrait l'identite d'Alice, cree un item DynamoDB avec status DRAFT. Repond : {"expenseId": "abc123", "status": "DRAFT"}

GetExpenses - Lister les notes

Recoit : token JWT. Fait : si employe, retourne ses notes. Si finance manager, retourne toutes les notes SUBMITTED. Repond : {"expenses": [...liste...]}

SubmitExpense - Soumettre une note

Recoit : expenseId dans l'URL + token JWT. Fait : verifie que la note appartient a Alice, change status de DRAFT a SUBMITTED. Repond : {"status": "SUBMITTED"}

ApproveExpense - Approuver une note

Recoit : expenseId + userId + token JWT de Bob. Fait : VERIFIE que Bob est dans le groupe 'finance', verifie SUBMITTED, change en APPROVED. Repond : {"status": "APPROVED"}

RejectExpense - Rejeter une note

Recoit : expenseId + userId + justification + token JWT de Bob. Fait : verification RBAC, change en REJECTED, sauvegarde la justification. Repond : {"status": "REJECTED"}

GetUploadUrl - Obtenir une URL d'upload

Recoit : expenseId + token JWT. Fait : verifie que la note appartient a Alice, genere une URL S3 pre-signee valide 10 minutes. Repond : {"uploadUrl": "https://s3.amazonaws.com/...?X-Amz-Signature=..."}

GetExpenseById - Obtenir une note precise

Reçoit : expenseld dans l'URL + token JWT. Fait : recupere l'item DynamoDB specifique, verifie les permissions. Repond : la note complete

3.3 Le langage C# et .NET 8

On a code le backend en C# avec .NET 8. **C# est un langage fortement type** : ca veut dire que le compilateur detecte les erreurs avant l'execution. Si tu passes un texte a la place d'un nombre, ca ne compile pas. **.NET 8 est supporte nativement par AWS Lambda** : Amazon fournit un runtime .NET 8 optimise pour Lambda. On utilise les memes competences cote frontend (MAUI) et backend (Lambda) : un seul langage pour tout le projet.

3.4 Structure d'une fonction Lambda

```
public class Function
{
    private readonly IAmazonDynamoDB _dynamo;
    private readonly string _tableName;

    // Constructeur : initialise les connexions AWS
    public Function()
    {
        _dynamo = new AmazonDynamoDBClient();
        _tableName = Environment.GetEnvironmentVariable("TABLE_NAME");
    }

    // Handler : point d'entree appele par AWS Lambda
    public async Task<APIGatewayProxyResponse> FunctionHandler(
        APIGatewayProxyRequest request, ILambdaContext context)
    {
        var claims = request.RequestContext.Authorizer.Claims;
        var userId = claims["sub"]; // identifiant unique Cognito

        var dto = JsonConvert.DeserializeObject<CreateExpenseDto>(request.Body);

        await _dynamo.PutItemAsync(new PutItemRequest {
            TableName = _tableName,
            Item = new Dictionary<string, AttributeValue> {
                ["PK"] = new() { S = "USER#" + userId },
                ["SK"] = new() { S = "EXPENSE#" + Guid.NewGuid() },
                ["status"] = new() { S = "DRAFT" }
            }
        });

        return new APIGatewayProxyResponse {
            StatusCode = 201,
            Body = "{\"message\": \"Expense created\"}"
        };
    }
}
```

3.5 Deployer une Lambda


```
# Se placer dans le dossier de la Lambda
cd backend/lambda/createExpense

# Deployer sur AWS
dotnet lambda deploy-function CreateExpense

# L'outil va :
# 1. Compiler le code C#
# 2. Créer un fichier ZIP avec le binaire
# 3. Uploader le ZIP sur AWS
# 4. Mettre à jour la fonction Lambda
```

Points clés :

Lambda = code qui s'exécute seulement quand nécessaire. On a 7 fonctions Lambda, chacune avec une responsabilité précise. Code en C# .NET 8, déployé avec 'dotnet lambda deploy-function'. Chaque Lambda lit les claims JWT pour connaître l'identité et le rôle de l'utilisateur.

Chapitre 4 - Amazon DynamoDB - La Base de Donnees NoSQL

4.1 SQL vs NoSQL - La difference fondamentale

Analogie :

Feuille Excel vs Dictionnaire : Une base SQL (MySQL, PostgreSQL), c'est comme une feuille Excel : des colonnes fixes, des lignes, des relations entre tables. DynamoDB (NoSQL), c'est comme un dictionnaire : chaque entree a une cle unique, et la valeur peut avoir n'importe quelle forme. Pas de schema fixe, pas de colonnes obligatoires.

Dans une base SQL, si tu veux stocker une note de frais, tu definis d'abord la structure :

```
-- SQL classique
CREATE TABLE expenses (
  id VARCHAR(36) PRIMARY KEY,
  user_id VARCHAR(36),
  amount DECIMAL(10,2),
  status VARCHAR(20),
  created_at TIMESTAMP
);
```

En DynamoDB, pas de schema : tu envoies directement un objet JSON et il est stocke tel quel. La flexibilite est totale, mais ca demande de reflechir differemment.

4.2 Notre table ExpenseReports

Notre table utilise le pattern **Single-Table Design** : une seule table stocke tous les types de donnees. La cle est composee de deux parties :

PK (Partition Key) : identifie le 'groupe' de donnees. Pour nous : USER#alice-uuid

SK (Sort Key) : identifie l'element dans le groupe. Pour nous : EXPENSE#expense-uuid

```
{
  "PK": "USER#a1b2c3d4-uuid-alice",
  "SK": "EXPENSE#x9y8z7-uuid-note",
  "expenseId": "x9y8z7-uuid-note",
  "userId": "a1b2c3d4-uuid-alice",
  "userEmail": "alice@test.com",
  "amount": 50.00,
  "category": "REPAS",
  "description": "Dejeuner client",
  "status": "SUBMITTED",
  "StatusGSI_PK": "STATUS#SUBMITTED",
  "createdAt": "2026-05-15T14:30:00Z",
  "updatedAt": "2026-05-15T15:00:00Z"
}
```

4.3 Le GSI (Global Secondary Index) - StatusIndex

Probleme : comment Bob (finance manager) peut voir toutes les notes SUBMITTED, venant de tous les employes ? Avec notre structure PK=USER#..., on ne peut requeter que les notes d'un seul utilisateur a la

fois.

Solution : le **GSI** (Global Secondary Index). C'est un index supplémentaire sur la table, avec une cle différente. Notre GSI 'StatusIndex' : Hash Key = StatusGSI_PK (ex: 'STATUS#SUBMITTED'), Range Key = SK. Quand Alice soumet sa note, on met a jour StatusGSI_PK = 'STATUS#SUBMITTED'. Bob peut alors requeter le GSI et recuperer TOUTES les notes soumises, de tous les employes.

```
// Requete du manager : toutes les notes SUBMITTED
var request = new QueryRequest {
    TableName = "ExpenseReports",
    IndexName = "StatusIndex",
    KeyConditionExpression = "StatusGSI_PK = :status",
    ExpressionAttributeValues = new Dictionary<string, AttributeValue> {
        [":status"] = new AttributeValue { S = "STATUS#SUBMITTED" }
    }
};
```

4.4 Query vs GetItem vs Scan

GetItem : recupere UN item precis par sa cle (PK + SK). Ultra rapide, latence inferieure a 5ms. On l'utilise pour GetExpenseById.

Query : recupere PLUSIEURS items avec la meme PK. Rapide, indexe. On l'utilise pour GetExpenses (toutes les notes d'Alice).

Scan : parcourt TOUTE la table. Lent et cher, a eviter absolument en production. On ne l'utilise JAMAIS.

Points cles :

DynamoDB est NoSQL : pas de schema fixe, flexibilite totale. Notre table utilise PK=USER#userId et SK=EXPENSE#expenseId. Le GSI StatusIndex permet au manager de trouver toutes les notes SUBMITTED sans scanner la table. Toujours utiliser GetItem ou Query, jamais Scan.

Chapitre 5 - Amazon API Gateway - La Porte d'Entree

5.1 C'est quoi une API REST ?

Analogie :

Le menu d'un restaurant : Une API REST, c'est comme le menu d'un restaurant. Tu (le client) regardes le menu, tu choisis un plat (endpoint), tu passes ta commande (requete HTTP), et le serveur (API Gateway) transmet a la cuisine (Lambda) qui prepare et renvoie le plat (reponse JSON).

REST signifie Representational State Transfer. C'est un style d'architecture pour les APIs web. Chaque action est definie par : une URL (l'endpoint) et un verbe HTTP (GET, POST, PUT, DELETE).

5.2 Les verbes HTTP expliqués

GET : lire des donnees. Exemple : GET /expenses -> lire toutes mes notes de frais

POST : creer quelque chose. Exemple : POST /expenses -> creer une nouvelle note

PUT : modifier quelque chose. Exemple : PUT /expenses/abc/submit -> soumettre la note abc

DELETE : supprimer. On n'en a pas dans notre projet.

5.3 Nos 7 endpoints

```
GET      /expenses                -> GetExpenses (liste)
POST     /expenses                -> CreateExpense (creation)
GET       /expenses/{expenseId}   -> GetExpenseById (detail)
POST     /expenses/{expenseId}/submit -> SubmitExpense
POST     /expenses/{expenseId}/approve -> ApproveExpense
POST     /expenses/{expenseId}/reject -> RejectExpense
GET      /expenses/{expenseId}/upload-url -> GetUploadUrl

Base URL: https://rwfqaety92.execute-api.eu-west-3.amazonaws.com/prod/
```

5.4 Le bug de l'URL que l'on a eu

On a eu un bug critique au debut : toutes les requetes retournaient 403 Forbidden. La cause etait subtile.

```
// MAUVAIS : BaseAddress sans '/' final
_http = new HttpClient {
    BaseAddress = new Uri("https://xxx.execute-api.eu-west-3.amazonaws.com/prod")
};
// _http.GetStringAsync("/expenses")
// -> https://xxx.execute-api.eu-west-3.amazonaws.com/expenses <- FAUX !

// CORRECT : BaseAddress avec '/' final + chemin sans '/'
_http = new HttpClient {
    BaseAddress = new Uri("https://xxx.execute-api.eu-west-3.amazonaws.com/prod/")
};
_http.GetStringAsync("expenses"); // -> /prod/expenses CORRECT
```

5.5 Le Cognito Authorizer

Chaque requete vers notre API doit contenir un token JWT valide dans le header HTTP. API Gateway verifie ce token automatiquement avant d'appeler Lambda.

```
// Dans l'app MAUI, on attache le token a chaque requete
_http.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", idToken);

// Ce que API Gateway recoit :
// Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...

// API Gateway verifie : signature valide ? token expire ?
// Si OK : transmet la requete a Lambda avec les claims du token
// Si KO : retourne 401 Unauthorized
```

Points clés :

API Gateway est le point d'entree unique de notre application. Les verbes HTTP definissent l'action : GET=lire, POST=creer. L'URL doit avoir un '/' final avec HttpClient. Le Cognito Authorizer verifie le JWT avant chaque appel Lambda.

Chapitre 6 - Amazon Cognito - L'Authentification

6.1 Le probleme de l'identite

Analogie :

Le badge d'entreprise : Quand tu entres dans une entreprise avec un badge, la securite verifie ton badge, voit ton nom et ton departement, et t'autorise a acceder a certaines zones. Cognito fait la meme chose pour notre app : il emet des 'badges numeriques' (tokens JWT) qui prouvent qui tu es et quel est ton role.

6.2 Le User Pool

Le User Pool Cognito est la liste de tous les utilisateurs de notre application. C'est lui qui gere : l'inscription et la connexion, la verification des mots de passe, les groupes d'utilisateurs.

Dans notre projet : User Pool ID = eu-west-3_IFDSxYrQK (region Paris).

Nos utilisateurs de test : alice@test.com (groupe employees), bob@test.com (groupe finance).

6.3 Le JWT (JSON Web Token)

Quand Alice se connecte, Cognito lui donne deux tokens JWT : IdToken et AccessToken. On utilise l'IdToken car il contient les informations dont on a besoin. Un JWT est compose de 3 parties separees par des points :

```
// Structure d'un JWT
eyJhbGciOiJSUzI1NiJ9      <- Header (encode en Base64)
.
eyJzdWIiOiJhbmwiYyZmIlLCJlbWVpYmCI6ImFsawNlQHRlc3QuY29tIn0
      <- Payload (encode en Base64)
.
SflKxwRJSMeKKF2QT4fwpM    <- Signature (verifie l'authenticite)

// Le Payload decode contient :
{
  "sub": "a1b2c3d4-uuid-alice",
  "email": "alice@test.com",
  "cognito:groups": ["employees"],
  "exp": 1747000000
}
```

6.4 Pourquoi IdToken et pas AccessToken ?

IdToken : contient sub, email, cognito:groups (les roles). C'est celui que nos Lambda lisent pour le RBAC.

AccessToken : contient sub et les scopes OAuth, mais PAS cognito:groups (selon la configuration).

Pour que Bob soit reconnu comme finance manager, Lambda doit lire cognito:groups dans le token. Donc on envoie l'IdToken.

6.5 Le RBAC dans le code Lambda

```
// Dans chaque Lambda qui necessite un role specifique
private static bool IsFinanceManager(IDictionary<string, string> claims)
{
    if (claims.TryGetValue("cognito:groups", out var groups))
        return groups.Contains("finance");
    return false;
}

// Utilisation dans ApproveExpense :
var claims = request.RequestContext.Authorizer.Claims;
if (!IsFinanceManager(claims))
{
    return new APIGatewayProxyResponse {
        StatusCode = 403,
        Body = "{\"error\": \"Only Finance Managers can approve.\"}"
    };
}
```

Points clés :

Cognito gere l'authentification et les groupes d'utilisateurs. Un JWT est un token signe qui prouve l'identite et le role. On utilise l'IdToken (et non l'AccessToken) car il contient cognito:groups. Lambda lit le claim cognito:groups pour le RBAC.

Chapitre 7 - Amazon S3 - Le Stockage des Recus

7.1 C'est quoi S3 ?

Analogie :

Google Drive pour developpeurs : S3 (Simple Storage Service) c'est comme Google Drive, mais pour les applications. Tu peux y stocker n'importe quel fichier (photos, videos, documents), y acceder via une URL, et configurer finement les permissions d'accès.

Notre bucket S3 : **expense-tracker-receipts-706922781773**. Il stocke les photos de recus des employes sous la structure : receipts/{userId}/{expenseId}.jpg

7.2 La Pre-Signed URL - Comment ca marche ?

Probleme : si on envoie la photo via Lambda, le fichier doit transiter par Lambda -> lent et couteux.

Solution : la **pre-signed URL**. Une pre-signed URL est une URL temporaire generee par Lambda (qui a les permissions S3) et donnee directement a l'app. L'app peut alors uploader la photo DIRECTEMENT sur S3, sans passer par Lambda.

```
// Lambda genere l'URL temporaire
var presignRequest = new GetPreSignedUrlRequest
{
    BucketName = "expense-tracker-receipts-706922781773",
    Key        = "receipts/" + userId + "/" + expenseId + ".jpg",
    Verb       = HttpVerb.PUT,
    Expires    = DateTime.UtcNow.AddMinutes(10),
    ContentType = "image/jpeg"
};
string uploadUrl = _s3.GetPreSignedURL(presignRequest);

// L'app MAUI upload directement sur S3
await uploadClient.PutAsync(uploadUrl, imageContent);
```

7.3 Securite du bucket

Notre bucket est completement prive. Personne ne peut y acceder directement depuis internet. Les seuls acces possibles sont : via une pre-signed URL generee par Lambda (valide 10 minutes max), ou directement par Lambda via son role IAM.

Points cles :

S3 stocke les photos de recus. La pre-signed URL permet a l'app d'uploader directement sur S3 sans passer par Lambda -> plus rapide et moins cher. Le bucket est prive : acces uniquement via URLs temporaires de 10 minutes.

Chapitre 8 - IAM - La Securite des Permissions

8.1 C'est quoi IAM ?

Analogie :

Les clefs d'un immeuble : IAM (Identity and Access Management) c'est le systeme de clefs et de badges d'un immeuble. Chaque employe a une clef qui lui donne acces uniquement aux pieces dont il a besoin. Le stagiaire ne peut pas entrer dans la salle serveur. Le directeur a acces partout. IAM fait la meme chose pour les services AWS.

8.2 Notre role Lambda

Notre role IAM **expense-tracker-lambda-role** est attache a toutes nos Lambda functions. Il leur donne exactement les permissions necessaires, et rien de plus.

```
// Permissions accordees au role Lambda
Autorisees :
dynamodb:GetItem      -> lire un item
dynamodb:PutItem      -> creer un item
dynamodb:UpdateItem   -> modifier un item
dynamodb:Query        -> requeter la table
s3:GetObject          -> lire un fichier S3
s3:PutObject          -> uploader un fichier S3
logs:CreateLogGroup   -> creer des logs CloudWatch
logs:PutLogEvents     -> ecrire des logs

Refusees (implicitement tout le reste) :
x dynamodb>DeleteItem -> supprimer des items
x s3>DeleteObject      -> supprimer des fichiers
x iam:*                -> modifier des permissions
x ec2:*                -> acceder aux serveurs
x rds:*                -> acceder aux bases SQL
```

Points clés :

IAM gere les permissions dans AWS. Le principe du moindre privilege : donner uniquement les acces necessaires. Notre Lambda ne peut faire que ce dont elle a besoin, rien de plus.

Chapitre 9 - Le Frontend - .NET MAUI et C#

9.1 C'est quoi .NET MAUI ?

MAUI signifie **Multi-platform App UI**. C'est un framework Microsoft qui permet de créer une seule application et de la déployer sur plusieurs plateformes : Windows, Android, iOS, macOS.

Dans notre projet : on cible Windows (application desktop) et Android (application mobile). On a une seule base de code C# qui tourne sur les deux.

9.2 Pourquoi MAUI et pas une app web ?

On aurait pu faire l'interface en React ou Vue.js (JavaScript). On a choisi MAUI parce que :

- On utilise déjà C# pour les Lambda -> même langage partout
- Accès natif aux fonctionnalités du téléphone (camera pour les reçus)
- Application qui tourne même hors ligne (mode brouillon)

9.3 XAML - Le langage de l'interface

Analogie :

HTML pour les apps : XAML (eXtensible Application Markup Language) est au frontend MAUI ce que HTML est aux pages web. C'est un langage XML qui décrit l'interface graphique : les boutons, les champs de texte, les listes. Le C# gère la logique, le XAML gère l'affichage.

Voici un exemple de notre LoginPage.xaml :

```
<!-- Un champ de saisie email dans XAML -->
<Border BackgroundColor="#F7FAFC"
        StrokeShape="RoundRectangle 8"
        Stroke="#CBD5E0">
    <Entry x:Name="EmailEntry"
           Placeholder="alice@company.com"
           TextColor="#1A202C"
           Keyboard="Email"
           HeightRequest="44"/>
</Border>

<!-- Un bouton -->
<Button Text="Se connecter"
        BackgroundColor="#FF9900"
        TextColor="white"
        Clicked="OnLoginClicked"/>
```

Et le code C# correspondant (LoginPage.xaml.cs) :

```
private async void OnLoginClicked(object sender, EventArgs e)
{
    string email    = EmailEntry.Text.Trim();
    string password = PasswordEntry.Text;

    var result = await _auth.LoginAsync(email, password);

    if (result.IsFinanceManager)
        await Shell.Current.GoToAsync("//ManagerPage");
    else
        await Shell.Current.GoToAsync("//EmployeePage");
}
```

9.4 Visual Studio 2026

Visual Studio est l'IDE qu'on utilise pour développer l'app MAUI. Il fournit : éditeur de code avec complétion automatique (IntelliSense), compilateur (transforme le C# en .exe), debugger (permet de mettre des points d'arrêt et inspecter les variables), designer XAML (aperçu visuel).

Le raccourci **F5** fait tout : compile le code, déploie l'app, la lance, et attache le debugger. Si tu mets un point d'arrêt dans le code, l'exécution se met en pause et tu peux inspecter les variables.

9.5 Le problème SDK qu'on a eu

On a eu un problème : Visual Studio 2026 utilise .NET 10 SDK par défaut, mais le workload MAUI n'était installé que sur .NET 9. VS ne reconnaissait pas le projet comme une app.

Solution : le fichier global.json qui force le SDK 9 :

```
// frontend/global.json
{
  "sdk": {
    "version": "9.0.203",
    "rollForward": "latestPatch"
  }
}
```

Points clés :

MAUI = une seule base de code C# pour Windows et Android. XAML décrit l'interface, C# gère la logique. Visual Studio compile et lance l'app avec F5. global.json force la version du SDK .NET.

Chapitre 10 - Structure du Code Frontend

10.1 AuthService.cs - La connexion

Ce service gere tout ce qui touche a l'authentification. Il utilise le SDK AWS Cognito pour .NET.

```
public class AuthService
{
    private static string? _idToken;
    private static bool _isFinanceManager;

    public async Task<LoginResult> LoginAsync(string email, string password)
    {
        var response = await _cognito.InitiateAuthAsync(new InitiateAuthRequest {
            AuthFlow = AuthFlowType.USER_PASSWORD_AUTH,
            ClientId = AppConfig.AppClientId,
            AuthParameters = new Dictionary<string, string> {
                { "USERNAME", email },
                { "PASSWORD", password }
            }
        });

        var jwt = new JwtSecurityTokenHandler()
            .ReadJwtToken(response.AuthenticationResult.IdToken);
        var groups = jwt.Claims
            .Where(c => c.Type == "cognito:groups")
            .Select(c => c.Value).ToList();

        _idToken = response.AuthenticationResult.IdToken;
        _isFinanceManager = groups.Contains("finance");

        return new LoginResult { Success = true, IsFinanceManager = _isFinanceManager };
    }
}
```

10.2 ExpenseService.cs - Les appels API

Ce service fait tous les appels HTTP vers notre API Gateway. C'est lui qui attache le token JWT a chaque requete.

```

public class ExpenseService
{
    private readonly HttpClient _http;

    public ExpenseService()
    {
        // IMPORTANT : le '/' final est obligatoire !
        _http = new HttpClient {
            BaseAddress = new Uri(
                "https://rwfqaety92.execute-api.eu-west-3.amazonaws.com/prod/")
        };
    }

    private void Auth()
    {
        var token = AuthService.GetIdToken();
        _http.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue("Bearer", token);
    }

    public async Task<List<ExpenseReport>> GetMyExpensesAsync()
    {
        Auth();
        var response = await _http.GetStringAsync("expenses");
        var json = JObject.Parse(response);
        return json["expenses"]?.ToObject<List<ExpenseReport>>()
            ?? new List<ExpenseReport>();
    }
}

```

10.3 Le modele ExpenseReport

Ce modele C# represente une note de frais. Il a des proprietes 'calculees' qui derivent automatiquement du status :

```

public class ExpenseReport
{
    public string ExpenseId { get; set; } = "";
    public decimal Amount { get; set; }
    public string Status { get; set; } = "";
    public string Category { get; set; } = "";

    // Proprietes calculees pour l'affichage
    public string StatusLabel => Status switch {
        "DRAFT" => "Brouillon",
        "SUBMITTED" => "En attente",
        "APPROVED" => "Approuvee",
        "REJECTED" => "Rejetee",
        _ => Status
    };

    public bool CanSubmit => Status == "DRAFT" || Status == "REJECTED";
    public bool CanReview => Status == "SUBMITTED";

    public string AmountFormatted => "EUR " + Amount.ToString("F2");
}

```

Points clés :

AuthService gere la connexion Cognito et stocke l'IdToken. ExpenseService fait tous les appels HTTP avec le token attache. Le modele ExpenseReport a des proprietes calculees qui controlent l'affichage selon le statut.

Chapitre 11 - Le Monitoring - CloudWatch

11.1 Pourquoi monitorer ?

Analogie :

Le tableau de bord d'une voiture : Tu n'attends pas que ta voiture tombe en panne pour regarder la jauge d'essence. Le tableau de bord te donne des infos en temps reel. CloudWatch est le tableau de bord de notre infrastructure AWS : invocations Lambda, erreurs, temps de reponse, tout est visible.

11.2 Notre dashboard

On a cree un dashboard CloudWatch **ExpenseTracker-Overview** avec 4 graphiques :

- Lambda Invocations (nombre d'appels par fonction)
- Lambda Errors (nombre d'erreurs)
- Lambda Duration (temps d'execution moyen en ms)
- API Gateway Requests (requetes totales, erreurs 4XX et 5XX)

11.3 L'alarme Lambda

On a configure une alarme qui se declenche si plus de 5 erreurs Lambda surviennent en 5 minutes.

```
# Configuration de l'alarme (AWS CLI)
aws cloudwatch put-metric-alarm \
  --alarm-name "ExpenseTracker-Lambda-Errors" \
  --metric-name Errors \
  --namespace AWS/Lambda \
  --threshold 5 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --period 300 \
  --evaluation-periods 1
```

11.4 DynamoDB PITR

PITR (Point-In-Time Recovery) est la sauvegarde automatique de DynamoDB. Elle conserve un historique de 35 jours. Si quelqu'un supprime accidentellement des donnees, on peut restaurer la table a n'importe quel moment dans cette fenetre.

Points clés :

CloudWatch surveille l'infrastructure en temps reel. Notre dashboard montre invocations, erreurs, duree et requetes API. L'alarme alerte si trop d'erreurs. DynamoDB PITR permet de restaurer les donnees sur 35 jours.

Chapitre 12 - Les Bugs Rencontres et Comment on les a Resolus

Tout projet rencontre des problemes. Voici les principaux bugs qu'on a resolu, et ce qu'on a appris.

12.1 Bug 1 - Erreur 403 sur tous les appels API

Symptome : Toutes les requetes vers l'API retournaient '403 Forbidden'. L'app se lancait, le login fonctionnait, mais impossible de charger les notes de frais.

Cause : HttpClient avec un BaseAddress sans '/' final.

```
// MAUVAIS
BaseAddress = "https://xxx.execute-api.eu-west-3.amazonaws.com/prod"
+ GetStringAsync("/expenses")
= https://xxx.execute-api.eu-west-3.amazonaws.com/expenses <- FAUX !

// CORRECT
BaseAddress = "https://xxx.execute-api.eu-west-3.amazonaws.com/prod/"
+ GetStringAsync("expenses")
= https://xxx.execute-api.eu-west-3.amazonaws.com/prod/expenses <- OK !
```

12.2 Bug 2 - Visual Studio dit 'bibliotheque de classes'

Symptome : VS refuse de lancer l'app avec l'erreur 'Impossible de demarrer directement un projet avec un type de sortie de bibliotheque de classes'.

Cause : VS evalue OutputType avant de charger les targets MAUI.

```
<!-- Solution : definir explicitement l'OutputType dans .csproj -->
<OutputType
  Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'windows'">
  WinExe
</OutputType>
```

12.3 Bug 3 - Mauvaise version de SDK

Symptome : VS 2026 utilisait .NET 10 SDK, mais le workload MAUI n'etait installe que sur .NET 9.

Solution : le fichier global.json force VS a utiliser le bon SDK.

```
// global.json place a la racine de la solution
{
  "sdk": {
    "version": "9.0.203",
    "rollForward": "latestPatch"
  }
}
```

12.4 Bug 4 - AccessToken vs IdToken

Symptome : Les utilisateurs du groupe 'finance' n'etaient pas reconnus comme managers.

Cause : on envoyait l'AccessToken a l'API, mais les claims cognito:groups ne sont pas garantis dans l'AccessToken. L'IdToken les contient toujours.

Solution : stocker et envoyer l'IdToken au lieu de l'AccessToken.

Points clés :

Lecon principale : les bugs viennent souvent de details subtils (un '/' manquant, un mauvais token).

La methodologie : reproduire le bug, comprendre la cause racine, corriger precisement, verifier que le build passe toujours.

Chapitre 13 - Glossaire - Les Termes Importants

AWS (Amazon Web Services)	La plateforme cloud d'Amazon. Propose des centaines de services accessibles via internet.
Lambda	Service AWS qui execute du code sans serveur. Le code s'execute uniquement quand declenche par un evenement.
Serverless	Architecture sans serveur gere. Le cloud s'occupe de tout, on paye a l'usage.
API REST	Interface de communication entre applications via HTTP. Utilise des URLs et des verbes (GET, POST, PUT, DELETE).
HTTP	Protocole de communication du web. Permet l'echange de donnees entre clients et serveurs.
JWT (JSON Web Token)	Token numerique signe qui prouve l'identite et le role d'un utilisateur. Compose de 3 parties : header, payload, signature.
Bearer Token	Format d'envoi du JWT dans le header HTTP : 'Authorization: Bearer '.
CORS	Cross-Origin Resource Sharing : mecanisme de securite qui controle quels domaines peuvent faire des requetes a un serveur.
NoSQL	Type de base de donnees sans schema fixe. Exemples : DynamoDB, MongoDB.
GSI (Global Secondary Index)	Index supplementaire sur une table DynamoDB qui permet de requeter selon des cles differentes de la cle principale.
PK / SK	Partition Key et Sort Key - les deux parties de la cle primaire d'un item DynamoDB.
RBAC	Role-Based Access Control : controle d'accès basé sur les rôles. Un utilisateur peut faire des actions selon son rôle.
SDK	Software Development Kit - boîte à outils pour développer avec un service spécifique.
IDE	Integrated Development Environment - logiciel pour coder (Visual Studio, VS Code).
XAML	Langage XML pour decire les interfaces graphiques MAUI.
Binding	Lien entre une propriete C# et un element XAML. Quand la propriete change, l'affichage se met a jour automatiquement.
Pre-signed URL	URL S3 temporaire et signee qui permet un acces specifique sans authentication.
IAM	Identity and Access Management - systeme de permissions AWS.
CloudWatch	Service AWS de monitoring. Collecte logs, metriques, et peut declencher des alarmes.
DynamoDB	Base de donnees NoSQL managee AWS. Haute performance, scalabilite automatique.
S3	Simple Storage Service - stockage de fichiers AWS.
Cognito	Service AWS d'authentification et de gestion des utilisateurs.
User Pool	Liste d'utilisateurs Cognito avec leurs attributs et groupes.
MAUI	Multi-platform App UI - framework Microsoft pour creer des apps cross-platform en C#.
.NET	Plateforme de developpement Microsoft. Supporte C#, F#, VB.NET.
C#	Langage de programmation oriente objet de Microsoft. Fortement type, moderne.
NuGet	Gestionnaire de paquets pour .NET (comme npm pour JavaScript).

Build	Compilation du code source en executable.
Deploy	Deploiement de l'application sur un environnement (AWS Lambda, Windows...).
Debug	Mode de demarrage avec le debugger attache, permettant d'inspecter le code en cours d'execution.